

PartyUp — Full Project Specification

Social Network for Finding Gaming Partners

1. Project Name

PartyUp — Game Partner Social Network

2. Project Type

PartyUp is a full-stack social network web application for gamers.

The system allows users to create profiles, search for games and gaming sessions, join or host sessions, publish posts, communicate through real-time chat, manage favorite games, view recent activity, and see dynamic statistics.

The project will be implemented using:

- React
 - Node.js
 - Express.js
 - MongoDB
 - Mongoose
 - MVC architecture
 - JQuery + Ajax
 - Socket.io / WebSockets
 - CSS3
 - Dynamic charts based on database data
-

3. Project Vision

Many gamers want to find suitable players to play with, but the process is often spread across different platforms, chats, communities, and social networks.

PartyUp provides one organized platform where users can find players based on game, platform, region, skill level, game mode, and availability.

The system turns the process of finding gaming partners into a structured social network experience that includes users, sessions, posts, chat, favorites, permissions, and statistics.

4. Main Goal

The main goal of PartyUp is to help users find gaming partners and manage gaming sessions in a social, organized, and interactive way.

The application should not be only a search tool. It should behave like a real social network that includes:

- Users
 - Gaming sessions
 - Posts
 - Feed
 - Join requests
 - Permissions
 - Favorites
 - Chat
 - Statistics
-

5. Target Users

The system is intended for:

1. **Regular gamers**
Users who want to find other players and join existing gaming sessions.
 2. **Session hosts**
Users who create gaming sessions and manage participants.
 3. **Group managers**
Users who manage private or public sessions and approve join requests.
 4. **Casual and competitive players**
Users who want to find teammates according to skill level, platform, region, and availability.
-

6. User Roles

The system will support two main roles.

6.1 Regular User

A regular user can:

- Register
- Log in
- Edit personal profile
- Search for games

- Search for sessions
- View public sessions
- Request to join private sessions
- Join public sessions
- View joined sessions
- Create posts
- Edit only their own posts
- Delete only their own posts
- View their personal feed
- Add games to favorites
- Remove games from favorites
- View recent activity
- Chat with users in joined sessions

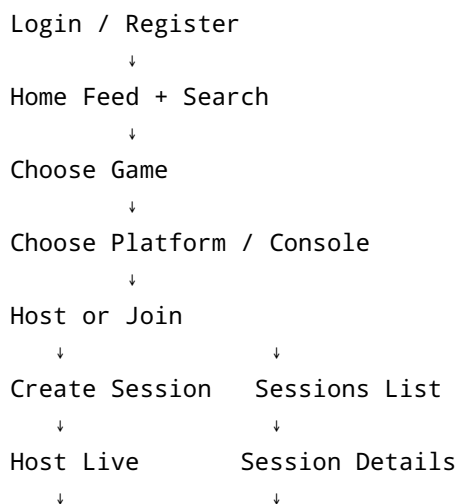
6.2 Host / Group Manager

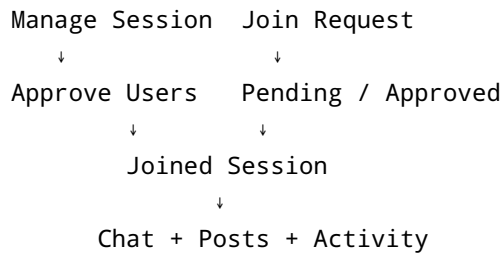
A host or group manager can do everything a regular user can do, and also:

- Create a new session
 - Edit sessions they created
 - Delete sessions they created
 - View join requests for their sessions
 - Approve join requests
 - Reject join requests
 - Manage session members
 - Remove users from a session
 - Publish host updates
 - Access management actions that regular users cannot access
-

7. Main User Flow

The main user flow of the system is:





8. Secondary Navigation

From the Home / Search page, the user can access:



9. Main Screens

9.1 Login Screen

Purpose:

Allow existing users to enter the system.

Fields:

- Username / email
- Password

Actions:

- Login
- Navigate to Register
- Show validation errors

Validations:

- Empty username
- Empty password
- Invalid credentials
- Server error

After successful login:

- The user is redirected to Home Feed + Search.
-

9.2 Register Screen

Purpose:

Allow new users to create an account.

Fields:

- Username
- Email
- Password
- Confirm password
- Region
- Platform
- Skill level
- Favorite games

Actions:

- Create account
- Navigate to Login

Validations:

- Required fields
- Password confirmation
- Duplicate username
- Duplicate email
- Invalid email format

After successful register:

- A new user is created in MongoDB.
 - The user is redirected to Home Feed + Search.
-

9.3 Home Feed + Search Screen

Purpose:

This is the main screen of the app.

The screen includes:

- Search bar
- Popular games section
- Personalized feed
- Suggested sessions

- Recent posts
- Navigation to Profile, Favorites, Recent Activity, My Sessions, and Statistics

The feed will display:

- Posts from sessions the user joined
- Posts related to the user's favorite games
- Public posts from public sessions
- Suggested sessions based on the user profile

Actions:

- Search games
 - Open game details
 - Open session details
 - Open post
 - Navigate to profile
 - Navigate to favorites
 - Navigate to recent activity
 - Navigate to statistics
-

9.4 Choose Game Screen

Purpose:

Allow the user to select a game.

The screen displays games such as:

- Valorant
- Fortnite
- Minecraft
- FIFA
- League of Legends
- Rocket League
- Call of Duty
- Apex Legends

Actions:

- Select a game
- Mark game as favorite
- Continue to platform selection

Data source:

- Games are loaded from MongoDB or predefined seed data.
-

9.5 Choose Console / Platform Screen

Purpose:

Allow the user to filter sessions by platform.

Platforms:

- PC
- PlayStation
- Xbox
- Nintendo Switch
- Mobile

Actions:

- Select platform
- Continue to Host / Join screen
- Toggle favorite game

This screen contributes to the advanced search filters.

9.6 Host / Join Screen

Purpose:

Allow the user to decide between creating a new session or joining an existing one.

Options:

1. Host

2. User creates a new gaming session.
3. User becomes the manager of that session.

4. Join

5. User views existing sessions.
6. User can filter and request to join.

Actions:

- Create session
 - View sessions list
 - Choose mode: Duo / Squad
-

9.7 Sessions List Screen

Purpose:

Show available sessions according to the selected filters.

Each session card includes:

- Session title
- Game
- Platform
- Region
- Skill level
- Mode: Duo / Squad
- Current players
- Max players
- Host name
- Public / Private status

Filters:

- Game
- Platform
- Region
- Skill level
- Mode
- Availability

Actions:

- View session details
- Search sessions
- Filter sessions
- Sort by recent / popular / available spots

This screen fulfills the advanced search requirement using at least three parameters.

9.8 Session Details Screen

Purpose:

Show full information about a selected session.

Displayed data:

- Session title
- Game name
- Platform
- Region
- Mode
- Skill level
- Description

- Host details
- Current members
- Max players
- Privacy status
- Recent posts
- Join button

Actions:

- Join public session
- Send join request to private session
- Open host profile
- View session posts
- Open chat if already a member

Authorization:

- Non-members cannot view private session content.
 - Members can view posts and chat.
 - Host can access management actions.
-

9.9 Create Session Screen

Purpose:

Allow users to create a new session.

Fields:

- Session title
- Game
- Platform
- Region
- Mode: Duo / Squad
- Skill level
- Max players
- Description
- Public / Private
- Start time / availability

Actions:

- Create session
- Cancel
- Return to previous screen

Validations:

- Required title
- Required game
- Required platform
- Max players must be valid

- Description length limit
- Skill level must be selected

After creation:

- A new session is saved in MongoDB.
- The creator becomes the host/manager.
- The user is redirected to Host Live / Manage Session.

9.10 Host Live / Manage Session Screen

Purpose:

Allow the host to manage a session.

The host can:

- View session details
- Edit session
- Delete session
- View pending join requests
- Approve users
- Reject users
- Remove members
- Publish updates/posts
- Open group chat
- View activity

Only the host can see management buttons.

Authorization examples:

- Regular users cannot approve members.
- Regular users cannot delete a session.
- A host can manage only sessions they created.

9.11 Join Request Screen / State

Purpose:

Handle requests to join private sessions.

Flow:

```
User clicks Join
      ↓
Join request is created
      ↓
Status: Pending
      ↓
```

Host approves or rejects



User receives updated status

Request statuses:

- Pending
- Approved
- Rejected

If approved:

- User becomes a session member.
- User can view private posts and chat.

If rejected:

- User remains outside the session.
- User cannot access private content.

9.12 Joined Session Screen

Purpose:

Show the active session after the user joins.

Displayed data:

- Session details
- Members list
- Posts
- Chat
- Activity log
- Leave session button

Actions:

- Write post
- Edit own post
- Delete own post
- Send chat message
- Leave session

Authorization:

- User can edit/delete only their own posts.
 - User can chat only if they are a session member.
-

9.13 Chat Screen

Purpose:

Real-time communication between session members.

Technology:

- Socket.io / WebSockets

Chat types:

- Group/session chat
- Optional user-to-user chat

Message data:

- Sender
- Session ID
- Message content
- Timestamp

Actions:

- Send message
- Receive messages in real time
- Load previous messages from MongoDB

Authorization:

- Only session members can access the session chat.
-

9.14 Profile Screen

Purpose:

Allow users to view and edit their profile.

Displayed data:

- Username
- Avatar
- Bio
- Region
- Platform
- Skill level
- Favorite games
- Joined sessions
- Created sessions
- User posts

Actions:

- Edit profile
- Save changes
- View own posts
- View own sessions

Authorization:

- User can edit only their own profile.
 - User cannot view private profile data of another user.
-

9.15 Favorites Screen

Purpose:

Show games marked as favorites.

Displayed data:

- Favorite games
- Related sessions
- Related posts

Actions:

- Remove game from favorites
 - Open game page
 - Search sessions for a favorite game
-

9.16 Recent Activity Screen

Purpose:

Show recent user activity.

Examples:

- Recently viewed games
 - Recently joined sessions
 - Recent posts
 - Recent chat activity
 - Recent join requests
-

9.17 My Sessions Screen

Purpose:

Show sessions related to the current user.

Sections:

- Sessions I joined
- Sessions I created
- Pending join requests
- Approved sessions
- Rejected requests

Actions:

- Open session
 - Manage session if host
 - Leave session
-

9.18 Statistics Dashboard

Purpose:

Display dynamic statistics from MongoDB.

The system will include at least two graphs.

Possible graphs:

1. Number of posts per month
2. Number of sessions per game
3. Number of users by platform
4. Number of join requests by status
5. Number of active sessions by region

The data must come from the database and reflect the current state of the system.

10. Data Models

The system will include at least three main models, plus additional supporting models.

10.1 User Model

Represents a registered user.

Fields:

- _id
- username
- email
- passwordHash
- avatarUrl
- bio

- region
- platform
- skillLevel
- favoriteGames
- role
- createdAt
- updatedAt

Relationships:

- User can create sessions.
- User can join sessions.
- User can create posts.
- User can send chat messages.
- User can create join requests.

CRUD operations:

- Create user
- Update user profile
- Delete user
- List users
- Search users

Search options:

- Search by username
- Search by region
- Search by platform
- Search by skill level
- Search by favorite game

Authorization:

- User can edit only their own profile.
- Host permissions are limited to related sessions.

10.2 Game Model

Represents a game in the system.

Fields:

- _id
- name
- genre
- imageUrl
- platforms
- description
- popularityScore

- createdAt
- updatedAt

CRUD operations:

- Create game
- Update game
- Delete game
- List games
- Search games

Search options:

- Search by name
- Search by platform
- Search by genre
- Search by popularity

Purpose:

- Used for search
 - Used for favorites
 - Used for session creation
 - Used for feed recommendations
-

10.3 Session / Group Model

Represents a gaming session or group.

Fields:

- _id
- title
- gameId
- hostId
- platform
- region
- mode
- skillLevel
- description
- maxPlayers
- members
- privacy
- status
- startTime
- createdAt
- updatedAt

Mode examples:

- Duo
- Squad
- Custom

Privacy:

- Public
- Private

Status:

- Open
- Full
- Closed

CRUD operations:

- Create session
- Update session
- Delete session
- List sessions
- Search sessions

Search options:

- Search by game
- Search by platform
- Search by region
- Search by skill level
- Search by mode
- Search by availability

Authorization:

- Only host can edit or delete the session.
- Only host can approve or reject join requests.
- Only members can access private session content.

10.4 Post Model

Represents a social post.

Fields:

- _id
- authorId
- sessionId
- content

- mediaUrl
- likes
- comments
- createdAt
- updatedAt

CRUD operations:

- Create post
- Update post
- Delete post
- List posts
- Search posts

Search options:

- Search by author
- Search by session
- Search by game
- Search by date
- Search by content keyword

Authorization:

- User can edit/delete only their own posts.
 - Users can view posts only if they have permission to view the related session.
 - Public posts can be visible in the general feed.
-

10.5 Join Request Model

Represents a request to join a private session.

Fields:

- _id
- userId
- sessionId
- status
- message
- createdAt
- updatedAt

Status values:

- Pending
- Approved
- Rejected

Operations:

- Create join request
- Update request status
- Delete request
- List requests
- Search requests

Authorization:

- User can create a request for themselves.
 - Host can approve or reject requests for their sessions.
 - User can view their own request status.
-

10.6 Chat Message Model

Represents a chat message.

Fields:

- _id
- sessionId
- senderId
- content
- createdAt

Operations:

- Create message
- List messages by session
- Delete message if needed
- Search messages if implemented

Authorization:

- Only session members can send/read messages.
-

11. Required CRUD Mapping

The system must allow the user to perform Create, Read/List, Update, Delete, and Search actions from the interface.

11.1 User CRUD

Create:

- Register screen creates a user.

List:

- User search results.
- Members list inside session.

Update:

- Edit Profile.

Delete:

- Delete account option or demo delete action.

Search:

- Search by username, platform, region, skill level, favorite game.
-

11.2 Game CRUD

Create:

- Add game through demo/admin action.

List:

- Game catalog / popular games.

Update:

- Edit game details through demo/admin action.

Delete:

- Delete game through demo/admin action.

Search:

- Search by name, platform, genre.
-

11.3 Session CRUD

Create:

- Create Session screen.

List:

- Sessions List / My Sessions.

Update:

- Host edits session.

Delete:

- Host deletes session.

Search:

- Search by game, platform, region, mode, skill level, availability.
-

11.4 Post CRUD

Create:

- User creates post inside joined session.

List:

- Home Feed / Session Posts / Profile Posts.

Update:

- User edits own post.

Delete:

- User deletes own post.

Search:

- Search by author, date, session, game, keyword.
-

11.5 Join Request CRUD

Create:

- User sends join request.

List:

- Host views pending requests / user views own requests.

Update:

- Host approves or rejects request.

Delete:

- User cancels request / host removes request.

Search:

- Search/filter by status, session, user.
-

11.6 ChatMessage CRUD

Create:

- User sends chat message.

List:

- Load previous messages by session.

Update:

- Optional edit message.

Delete:

- Optional delete message.

Search:

- Optional search messages by content/session/date.
-

12. Advanced Search Requirements

The system will include at least two advanced search features where the user can choose at least three parameters.

12.1 Advanced Session Search

Parameters:

- Game
- Platform
- Region
- Skill level
- Mode
- Availability

Example:

```
Game: Valorant
Platform: PC
Region: Israel
Skill Level: Intermediate
Mode: Squad
```

Result:

- The system returns matching sessions from MongoDB.
-

12.2 Advanced User Search

Parameters:

- Username
- Favorite game
- Platform
- Region
- Skill level

Example:

```
Favorite Game: Fortnite
Platform: PlayStation
Region: Israel
Skill Level: Beginner
```

Result:

- The system returns matching users from MongoDB.
-

13. Authorization Rules

The system must protect pages and actions according to the logged-in user.

Rules:

1. Guest users cannot access the main app.
2. Logged-in users can access public content.
3. Users can edit only their own profile.
4. Users can edit/delete only posts they created.
5. Users can delete only their own join requests.
6. Hosts can edit/delete only sessions they created.
7. Hosts can approve/reject requests only for their own sessions.
8. Users cannot see private session posts unless they are approved members.
9. Users cannot access chat unless they are session members.

10. Management buttons appear only for the host.

14. Feed Logic

The Home Feed will include:

- Posts from sessions the user joined
- Public posts from public sessions
- Posts related to favorite games
- Suggested sessions based on profile preferences
- Recent activity from joined sessions

Private content rule:

- Private session posts appear only for approved members.
-

15. Favorites Logic

Users can mark games as favorites.

Favorites are used for:

- Profile display
- Feed recommendations
- Quick access to sessions
- Personalized suggestions

Actions:

- Add game to favorites
 - Remove game from favorites
 - View favorite games
 - View sessions related to favorite games
-

16. Recent Activity Logic

The system will track or display recent user-related activity:

- Recently viewed games
- Recently joined sessions
- Recently created posts
- Recent join request status
- Recent chat activity

This makes the app feel more like a real social network.

17. Chat Logic

The chat will use Socket.io or WebSockets.

Flow:

```
User enters joined session
      ↓
Client connects to Socket.io room by session ID
      ↓
User sends message
      ↓
Server receives message
      ↓
Server saves message in MongoDB
      ↓
Server broadcasts message to session members
      ↓
Clients display message in real time
```

Rules:

- Only session members can enter the chat room.
 - Messages are stored in MongoDB.
 - Previous messages are loaded when opening chat.
 - New messages appear in real time.
-

18. Statistics Logic

The Statistics Dashboard will show at least two dynamic graphs.

The graphs will be based on MongoDB data.

Graph 1:

Posts per month

Purpose:

- Show how many posts were created each month.

Data source:

- Post collection.

Graph 2:

Sessions per game

Purpose:

- Show how many sessions exist for each game.

Data source:

- Session collection joined with Game data.

Optional graph 3:

Users by platform

Purpose:

- Show distribution of users by platform.

Data source:

- User collection.

Optional graph 4:

Join requests by status

Purpose:

- Show pending / approved / rejected requests.

Data source:

- JoinRequest collection.

19. React Requirements

The application will use regular React for the client-side interface.

React responsibilities:

- Screen rendering
- Navigation between screens
- State management
- Forms
- Session cards
- Feed posts
- Profile page
- Favorites
- Chat UI
- Statistics dashboard

Required React features:

1. Video

2. A video component can appear in the game/session details screen.

3. Example: game trailer, session intro, or demo media.

4. Canvas

5. A canvas component can be used for visual badges, player card preview, or simple animated gaming background.

20. JQuery and Ajax Requirements

The project will include JQuery and Ajax usage.

JQuery/Ajax will be used for selected client-server requests, for example:

- Loading games list
- Searching sessions
- Submitting advanced search
- Loading statistics
- Updating favorites
- Sending join request

Example flow:

```
User selects filters
      ↓
JQuery Ajax sends GET request to server
      ↓
Server queries MongoDB
      ↓
Response returns JSON
      ↓
Client updates the UI
```

21. CSS3 Requirements

The system will use the required CSS3 features:

1. Text-shadow

2. Used in titles, game cards, or hero text.

3. Transition

4. Used for buttons, cards, hover effects, modal transitions.

5. Multiple columns

6. Used in profile details, game information, or statistics text sections.

7. Font-face

8. Used to load a custom font for the app branding.

9. Border-radius

10. Used widely in cards, buttons, inputs, modals, and mobile-style UI.

22. MVC Architecture

The backend will be organized using MVC.

22.1 Models

Responsible for MongoDB schemas.

Examples:

- User.js
 - Game.js
 - Session.js
 - Post.js
 - JoinRequest.js
 - ChatMessage.js
-

22.2 Controllers

Responsible for business logic.

Examples:

- authController.js
- userController.js
- gameController.js
- sessionController.js
- postController.js
- requestController.js
- chatController.js

- statsController.js
-

22.3 Routes

Responsible for API endpoints.

Examples:

- authRoutes.js
 - userRoutes.js
 - gameRoutes.js
 - sessionRoutes.js
 - postRoutes.js
 - requestRoutes.js
 - chatRoutes.js
 - statsRoutes.js
-

22.4 Views / Client

Responsible for UI and user interaction.

Examples:

- React screens
 - JQuery/Ajax calls
 - CSS
 - HTML
-

23. API Endpoints

23.1 Auth Endpoints

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET  /api/auth/me
```

23.2 User Endpoints

```
POST  /api/users
GET   /api/users
GET   /api/users/search
```

```
GET    /api/users/:id
PUT    /api/users/:id
DELETE /api/users/:id
```

23.3 Game Endpoints

```
POST   /api/games
GET    /api/games
GET    /api/games/search
GET    /api/games/:id
PUT    /api/games/:id
DELETE /api/games/:id
```

23.4 Session Endpoints

```
POST   /api/sessions
GET    /api/sessions
GET    /api/sessions/search
GET    /api/sessions/:id
PUT    /api/sessions/:id
DELETE /api/sessions/:id
POST   /api/sessions/:id/join
POST   /api/sessions/:id/leave
```

23.5 Post Endpoints

```
POST   /api/posts
GET    /api/posts
GET    /api/posts/search
GET    /api/posts/:id
PUT    /api/posts/:id
DELETE /api/posts/:id
```

23.6 Join Request Endpoints

```
POST   /api/requests
GET    /api/requests/my
GET    /api/requests/session/:sessionId
PUT    /api/requests/:id/approve
```

```
PUT    /api/requests/:id/reject
DELETE /api/requests/:id
```

23.7 Chat Endpoints

```
GET /api/chat/:sessionId/messages
```

Socket.io events:

```
joinRoom
sendMessage
receiveMessage
leaveRoom
```

23.8 Statistics Endpoints

```
GET /api/stats/posts-per-month
GET /api/stats/sessions-per-game
GET /api/stats/users-by-platform
GET /api/stats/requests-by-status
```

24. Database Seed Data

Before the defense, the database will contain realistic demo data.

Seed data will include:

- At least 5 users
 - At least 8 games
 - At least 10 sessions
 - Several public sessions
 - Several private sessions
 - Posts for different sessions
 - Join requests with different statuses
 - Chat messages
 - Favorite games per user
-

24.1 Demo Users

User	Role	Platform	Region	Skill Level	Favorite Games
Menalu	Regular User	PC	Israel	Intermediate	Valorant, Fortnite
Niv	Host	PlayStation	Israel	Advanced	FIFA, Rocket League
Trngo	Regular User	PC	Europe	Beginner	Minecraft, Fortnite
Mosh	Regular User	Xbox	Israel	Intermediate	Call of Duty, Apex Legends
Noa	Host	PC	Israel	Advanced	Valorant, League of Legends

24.2 Example Games

- Valorant
- Fortnite
- Minecraft
- FIFA
- League of Legends
- Rocket League
- Apex Legends
- Call of Duty

24.3 Example Sessions

- Valorant Ranked Squad
- Fortnite Casual Duo
- Minecraft Survival Team
- FIFA Weekend Match
- Rocket League 2v2
- Apex Legends Night Squad
- Call of Duty Tactical Team
- League of Legends Ranked Flex

25. Error Handling

The system will handle errors both on the client side and server side.

Client-side errors:

- Empty required fields
- Invalid email format
- Password mismatch
- Missing search filters
- Invalid form values

Server-side errors:

- Duplicate username
- Invalid login
- Unauthorized action
- Forbidden private content
- Invalid MongoDB ID
- Session is full
- Join request already exists
- User is already a member
- Server/database error

The server must not crash when receiving invalid input.

26. Validation Examples

Examples:

1. User tries to register without username
→ show error: "Username is required."
 2. User tries to join a full session
→ show error: "This session is already full."
 3. User tries to delete another user's post
→ return forbidden error.
 4. User tries to access private session without approval
→ return unauthorized/forbidden response.
 5. Host tries to approve a request for a session they do not own
→ return forbidden error.
-

27. Security Considerations

The system will include basic security practices:

- Passwords will not be stored as plain text.
- Passwords will be hashed before saving.
- Sensitive actions will require authentication.
- Authorization checks will be done on the server.
- Private session data will be protected.
- Environment variables will be used for sensitive configuration.

No API keys, passwords, tokens, or sensitive information should be uploaded to GitHub.

28. Demo Scenario for Defense

The project will include a prepared demo flow.

Demo regular user:

Username: Menalu
Password: demo123

Demo host user:

Username: Noa
Password: demo123

Demo flow:

1. Login as **Menalu**.
2. View Home Feed.
3. Search for **Valorant** sessions.
4. Filter by **PC + Israel + Intermediate + Squad**.
5. Open a private session hosted by **Noa**.
6. Menalu sends Join Request.
7. Logout or switch to host user.
8. Login as **Noa**.
9. Open Host Live / Manage Session.
10. Noa approves Menalu's request.
11. Return to Menalu.
12. Open Joined Session.
13. Menalu writes a post.
14. Menalu sends a chat message.
15. Open Statistics Dashboard.
16. Show dynamic graphs.

This demo covers:

- Login
 - Search
 - Session list
 - Session details
 - Join request
 - Host approval
 - Permissions
 - Posts
 - Chat
 - Statistics
-

29. Suggested Folder Structure

```
partyup/
├── server/
│   ├── app.js
│   ├── server.js
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   ├── User.js
│   │   ├── Game.js
│   │   ├── Session.js
│   │   ├── Post.js
│   │   ├── JoinRequest.js
│   │   └── ChatMessage.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── userController.js
│   │   ├── gameController.js
│   │   ├── sessionController.js
│   │   ├── postController.js
│   │   ├── requestController.js
│   │   ├── chatController.js
│   │   └── statsController.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── userRoutes.js
│   │   ├── gameRoutes.js
│   │   ├── sessionRoutes.js
│   │   ├── postRoutes.js
│   │   ├── requestRoutes.js
│   │   ├── chatRoutes.js
│   │   └── statsRoutes.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   ├── errorMiddleware.js
│   │   └── validateMiddleware.js
│   └── seed/
│       └── seedData.js
└── client/
    └── index.html
```

```
├── src/
│   ├── app.jsx
│   ├── main.jsx
│   ├── auth.jsx
│   ├── screens.jsx
│   ├── api/
│   │   └── apiClient.js
│   ├── components/
│   │   ├── BottomNav.jsx
│   │   ├── SessionCard.jsx
│   │   ├── PostCard.jsx
│   │   ├── ChatBox.jsx
│   │   ├── StatsChart.jsx
│   │   ├── VideoPreview.jsx
│   │   └── PlayerCanvasCard.jsx
│   └── styles/
│       └── style.css
├── package.json
├── README.md
└── .env.example
```

Requirement	PartyUp Implementation
Social network	Users, sessions, posts, feed, favorites, chat, activity
Node.js + Express	Backend server and REST API
MongoDB	Data storage for users, games, sessions, posts, requests, messages
MVC	Models, controllers, routes, client views
At least 3 models	User, Game, Session, Post, JoinRequest, ChatMessage
CRUD for each model	Create, list, update, delete, search through UI and API
Advanced search	Session search and user search using at least three parameters
Permissions	Regular user vs host, private session access, edit/delete restrictions
Feed	Personalized feed based on joined sessions and favorite games
Demo data	Menalu, Niv, Trngo, Mosh, Noa + games, sessions, posts, messages
Validation and error handling	Client-side and server-side handling
JQuery + Ajax	Used for selected dynamic API calls

Requirement	PartyUp Implementation
React	Used for main UI screens and components
Video and Canvas	Included in game/session UI
CSS3	Text-shadow, transition, multiple columns, font-face, border-radius
Socket.io / WebSockets	Real-time session chat
Dynamic graphs	At least two charts based on MongoDB data

31. Final Project Summary

PartyUp is a full-stack social network for gamers who want to find teammates and manage gaming sessions.

The system includes authentication, user profiles, game catalog, session creation, session joining, join requests, host approval, posts, personalized feed, favorites, recent activity, real-time chat, permissions, search, statistics, and dynamic data from MongoDB.

The project is designed to meet the Android 2 final project requirements while keeping the original PartyUp visual identity and user experience.